



Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 10

Laporan Jobsheet

Praktikum 1

```
1. [id].tsx M X
src > pages > produk > [id].tsx > DetailProdukPage
6  const DetailProdukPage = () => {
31  }
32
33
34  return (
35    <div className={styles.produk_detail} style={{ padding: "50px", textAlign: "center" }}>
36      <h1>Detail Produk</h1>
37      <hr style={{ margin: "20px 0" }} />
38
39      <div style={{ display: "flex", gap: "30px", justifyContent: "center", alignItems: "center" }}>
40        <img
41          src={product.images || "https://placeholder.co/400x400?text=No+Image"}
42          alt={product.name}
43          style={{ width: "400px", borderRadius: "10px", boxShadow: "0 4px 8px rgba(0,0,0,0.1)" }}
44        />
45
46        <div style={{ textAlign: "left", maxWidth: "500px" }}>
47          <h2 style={{ fontSize: "2rem", marginBottom: "10px" }}>{product.name}</h2>
48          <p style={{ color: "#666", fontSize: "1.2rem" }}>Kategori: {product.category}</p>
49          <p style={{ color: "#666", fontSize: "1.2rem" }}>ID Barang: {product.id}</p>
50
51          <h3 style={{ color: "#000", fontSize: "1.5rem", marginTop: "20px" }}>
52            Harga: Rp {Number(product.price || 0).toLocaleString("id-ID")}
53          </h3>
54        </div>
45      </div>
35    </div>
34  </return (
33  </return (
32  </return (
31  </return (
6  </const DetailProdukPage = () => {
31  </>
32  </>
33  </>
34  </return (
35  </div className={styles.produk_detail} style={{ padding: "50px", textAlign: "center" }}>
36  </h1>Detail Produk</h1>
37  </hr style={{ margin: "20px 0" }} />
38  </div style={{ display: "flex", gap: "30px", justifyContent: "center", alignItems: "center" }}>
39  </img
40  </src={product.images || "https://placeholder.co/400x400?text=No+Image"}
41  </alt={product.name}
42  </style={{ width: "400px", borderRadius: "10px", boxShadow: "0 4px 8px rgba(0,0,0,0.1)" }}
43  </>
44  </div style={{ textAlign: "left", maxWidth: "500px" }}>
45  </h2 style={{ fontSize: "2rem", marginBottom: "10px" }}>{product.name}</h2>
46  </p style={{ color: "#666", fontSize: "1.2rem" }}>Kategori: {product.category}</p>
47  </p style={{ color: "#666", fontSize: "1.2rem" }}>ID Barang: {product.id}</p>
48  </h3 style={{ color: "#000", fontSize: "1.5rem", marginTop: "20px" }}>
49  </Harga: Rp {Number(product.price || 0).toLocaleString("id-ID")}
50  </>
51  </>
52  </>
53  </>
54  </>
```

ID Barang: 9mIrrd07IIEUD1r0gsBX

- Melakukan editing pada file id sehingga bisa meretur id barang nya



2.

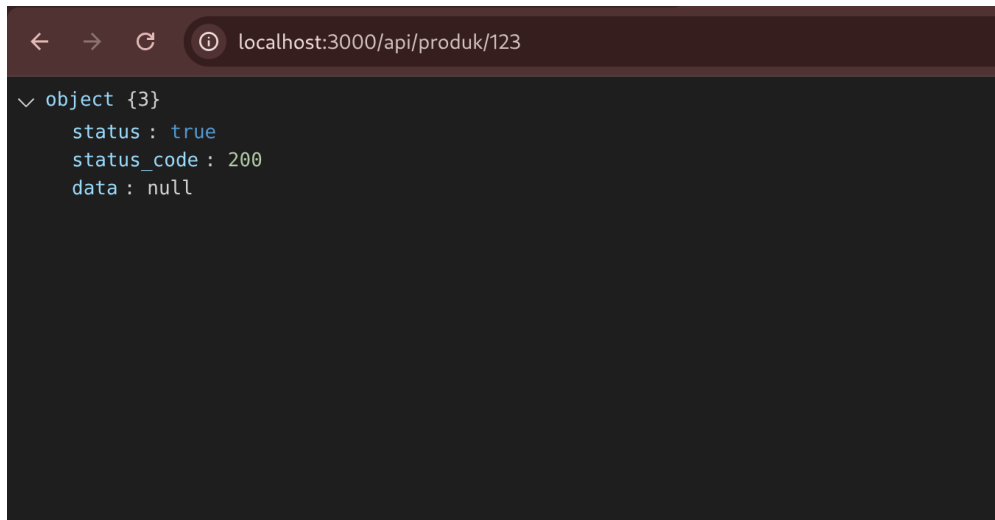
```
1 import { useRouter } from "next/router";
2 import useSWR from "swr";
3 import fetcher from "../../utils/swr/fetcher";
4 import styles from "@views/produk/produk.module.scss";
5
6 export default function DetailProdukPage () {
7   // const Router = useRouter();
8   // console.log(Router);
9   const { query } = useRouter();
10  const {data, error, isLoading} = useSWR(`/api/produk/${query.id}`, fetcher);
11
12  return (
13    <div>
14      <h1>Halaman Produk</h1>
15      <p>Produk: {query.id}</p>
16    </div>
17  )
18 };
19
20
```

```
1 import type { NextApiRequest, NextApiResponse } from "next";
2 import { retrieveProducts, retrieveProductByID } from "../../utils/db/servicefirebase"
3
4 export default async function handler(req: NextApiRequest, res: NextApiResponse) {
5
6   if (req.query.produk![1]) {
7     const data = await retrieveProductByID("products", req.query.produk![1]);
8     res.status(200).json({ status: true, status_code: 200, data });
9     return;
10  } else {
11    const data = await retrieveProducts("products");
12    res.status(200).json({ status: true, status_code: 200, data });
13  };
14 }
```



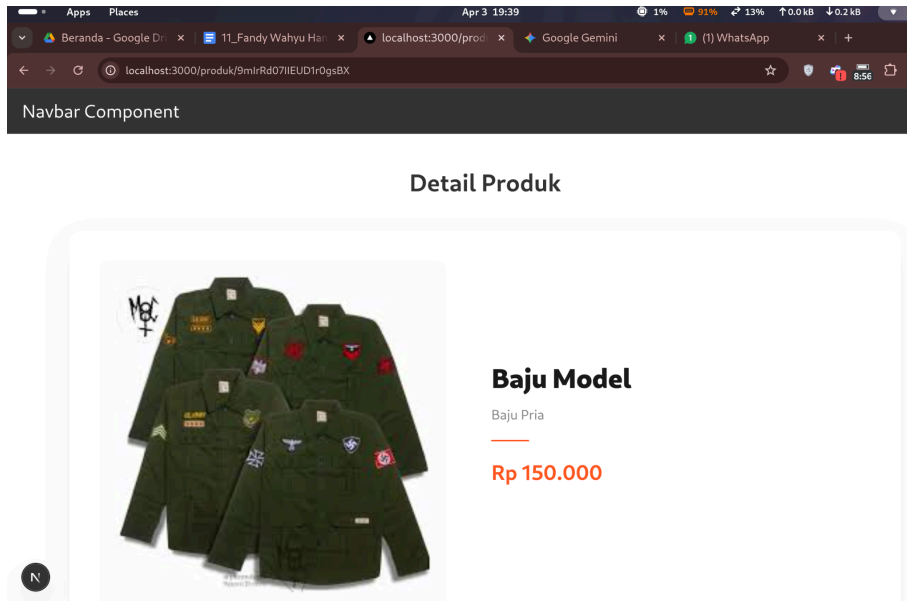
```
1 import { getFirestore, collection, getDocs, getDoc, doc } from "firebase/firestore";
2 import app from "../firebase";
3 const db = getFirestore(app);
4
5 export async function retrieveProducts(collectionName: string) {
6   const snapshot = await getDocs(collection(db, collectionName));
7   const data = snapshot.docs.map((doc) => ({
8     id: doc.id,
9     ...doc.data(),
10  }));
11   return data;
12 }
13
14 export async function retrieveProductByID(collectionName: string, id: string) {
15   const docRef = doc(db, collectionName, id);
16   const snapshot = await getDoc(docRef);
17   const data = snapshot.data();
18
19   if (data) {
20     return { id: snapshot.id, ...data };
21   }
22   return null;
23 }
```

```
localhost:3000/api/produk/DUGF726E1JRMG7ZNM8sQ
{
  status: true,
  status_code: 200,
  data: {
    id: "DUGF726E1JRMG7ZNM8sQ",
    category: "Baju Anak",
    price: 10000,
    name: "Baju polo",
    images: "https://www.johnbullclothing.com/cdn/shop/products/kombat-kids-army-tank-suit-btp-joh",
    size: "XL"
  }
}
```



```
1 .produkDetail {
2   }
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84 }
85
86 @media (max-width: 768px) {
87   .produkDetail {
88     flex-direction: column;
89     padding: 20px;
90     margin: 20px;
91     gap: 30px;
92
93     &__name { font-size: 28px; }
94   }
95 }
96
97 .title {
98   text-align: center;
99   font-size: 32px;
100  font-weight: 700;
101  color: #333;
102  margin-top: 40px;
103 }
```

```
1 import { ProductType } from "../../types/Product.types";
2 import styles from "@views/DetailProduk/Detailproduk.module.scss";
3
4 export default function DetailProduk({ products }: { products: ProductType }) {
5   return (
6     <div>
7       <h1 className={styles.title}>Detail Produk</h1>
8       <div className={styles.produkDetail}>
9         <div className={styles.produkDetail_image}>
10          <img
11            src={products.images || "https://placeholder.co/400x400?text=No+Image"}
12            alt={products.name}
13          />
14        </div>
15
16        <div className={styles.produkDetail_info}>
17          <h1 className={styles.produkDetail_name}>{products.name}</h1>
18          <p className={styles.produkDetail_category}>{products.category}</p>
19          <div className={styles.produkDetail_divider} />
20          <p className={styles.produkDetail_price}>
21            Rp {Number(products.price || 0).toLocaleString("id-ID")}
22          </p>
23        </div>
24      </div>
25    </div>
26  )
27 }
```



- Mengubah api produk lalu menambahkan detail pada halaman detail barang ketika di klik akan menghasilkan halaman

```
18
19 export async function getServerSideProps({params} : {params: {id: string}}){
20   const res = await fetch(`http://localhost:3000/api/produk/${params.id}`);
21   const response = await res.json();
22
23   return {
24     props: {
25       product: response.data
26     }
27   }
28 }
29
30
```

3.

- Menambahkan fitur ssr untuk mempercepat saat masuk ke dalam halaman detail produk



```
20 export async function getStaticPaths() {
21   const res = await fetch('http://localhost:3000/api/produk');
22   const data = await res.json();
23
24   const paths = data.data.map((produk: any) => ({
25     params: { id: produk.id },
26   }));
27
28   return { paths, fallback: false };
29 }
30
31
32 export async function getStaticProps({params} : {params: {id: string}}) {
33   const res = await fetch(`http://localhost:3000/api/produk/${params.id}`);
34   const response : {data: ProductType} = await res.json();
35
36   return {
37     props: {
38       product: response.data
39     }
40   }
41 }
```

4.

- Menambahkan get static path dan props agar data bisa tampil langsung secara statis

Tugas Praktikum

1.

Aspek	Client-Side Rendering (CSR)	Server-Side Rendering (SSR)	Static Site Generation (SSG)
Loading	Lambat di awal (initial load), cepat setelahnya.	Cepat untuk tampilan awal di sisi user.	Sangat cepat (karena file sudah jadi HTML statis).



Build Required	Tidak perlu build ulang saat data berubah.	Tidak perlu build ulang (dirender tiap ada request).	Ya , perlu build ulang (kecuali pakai ISR).
SEO	Kurang optimal (konten dirender JavaScript).	Sangat baik (konten sudah ada di HTML awal).	Sangat baik (konten sudah tersedia sejak build).
Perubahan Data	Sangat dinamis (cocok buat dashboard/admin).	Dinamis (diambil langsung dari server tiap request).	Statis/Tetap (cocok buat blog atau dokumentasi).

Tugas Praktikum

1. Mengapa `getStaticPaths` wajib pada dynamic SSG?

- Pada metode SSG (Static Site Generation), semua halaman `.html` harus dibuat/dirender pada saat Build Time (pas lu ngetik `npm run build`). Karena URL-nya dinamis (pake `[id]`), Next.js nggak tahu ada ID apa saja di database lu.

2. Mengapa CSR membutuhkan loading state?

- Pada CSR (Client-Side Rendering), saat browser pertama kali buka halaman, server cuma ngasih file HTML kosong dan JavaScript. Data dari API baru ditarik setelah halaman muncul di layar (pake `useEffect` atau `useSWR`).

3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?

- Ini karena sifat dasar SSG adalah Static. Begitu proses *build* selesai, Next.js udah "bungkus" data dari database jadi file HTML mati.



4. Mana metode terbaik untuk halaman detail e-commerce?

- SSG + ISR (Rekomendasi Utama): Sangat cepat (SEO kencang) dan bisa update data secara otomatis setiap beberapa detik tanpa build ulang manual.
- SSR: Kalau harga produk berubah setiap detik (kayak harga emas atau saham) dan harus selalu *real-time* saat halaman di-refresh.
- CSR: Kurang disarankan buat detail produk karena SEO-nya jelek (Google Bot susah nangkap info produk kalau datanya baru muncul belakangan).

5. Apa risiko menggunakan SSG untuk produk yang sering berubah?

- Stale Data (Data Basi): User bisa ngeliat harga lama atau stok yang aslinya udah abis tapi di layar masih ada (karena belum di-build ulang).
- Inkonsistensi: Ada perbedaan data antara halaman list produk (yang mungkin pake CSR/SWR) dengan halaman detail (yang pake SSG).
- Build Time Membengkak: Kalau lu punya 10.000 produk dan tiap ada perubahan harus *rebuild* total, prosesnya bakal makan waktu lama dan boros sumber daya server.